

ECE4760 Lab 2: Digital Galton Board

Stephen Barlett, Jamayne Gyimah-Danquah, Md Shad

October 23, 2025

1 Introduction

In this lab, we implemented a digital Galton Board on the RP2040 microcontroller to model the statistical behavior of balls undergoing repeated binary interactions. Each ball fell through a triangular array of pegs, with every collision representing a Bernoulli trial that caused the ball to deflect left or right with equal probability. As more collisions occurred, the resulting distribution of balls across the bins at the bottom of the board approximated the Gaussian curve predicted by the Central Limit Theorem. Each collision triggered a “thunk” sound generated by a Direct Memory Access (DMA) channel streaming samples to an SPI-connected MCP4822 DAC, minimizing CPU overhead for audio output.

The VGA display rendered the Galton Board and a real-time histogram showing the distribution of outcomes. It also displayed key parameters such as the number of active balls, total balls dropped, bounciness coefficient, and elapsed time. A potentiometer and pushbutton provided user control, with the potentiometer adjusting either the number of balls or the coefficient of restitution depending on the current mode. Each adjustment reset the histogram to allow for new trials under different conditions.

Although we ran the program entirely on a single RP2040 core, careful optimization of arithmetic routines, data structures, and DMA operations ensured that the system was neither performance nor memory constrained. The final implementation successfully visualized the emergence of a Gaussian distribution from simple random processes while demonstrating efficient use of numerical computation, real-time rendering, and peripheral integration on a resource-limited embedded platform.

2 Design and Testing

2.1 Overview

The hardware design and testing of the digital Galton Board involved integrating multiple subsystems — VGA display output, audio generation via DAC, potentiometer-based control input, and a button interface for user mode toggling. The entire system was built around the Raspberry Pi Pico microcontroller. The design process followed a structured, incremental development approach — beginning with hardware assembly and basic signal verification, followed by integration and final validation.

Each subsystem was tested both in isolation and as part of the integrated system to ensure performance consistency, reliability, and full compliance with the lab requirements. The end goal was to achieve a visually and aurally responsive simulation at 30 frames per second (fps), where balls visually bounce off pegs while producing synchronized “thunk” audio tones. The user could dynamically adjust the number of balls and bounciness using a potentiometer, toggling between parameters with a debounced button press.

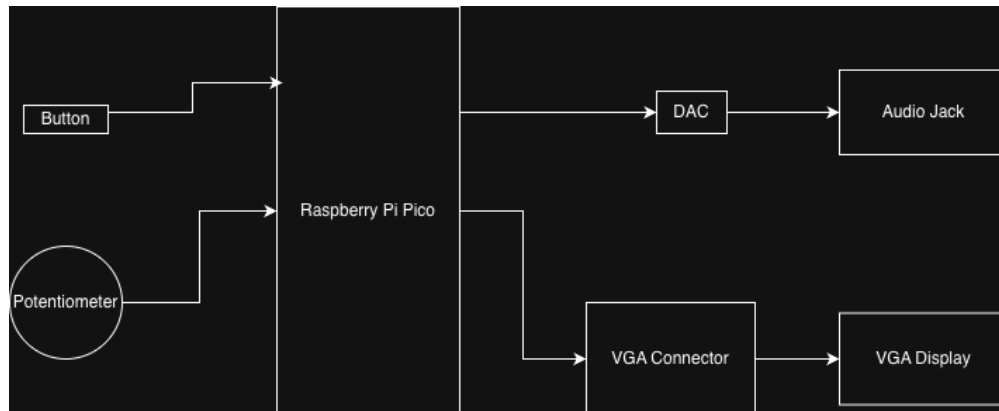


Figure 1: High-level hardware setup showing the Raspberry Pi Pico connected to the VGA circuit, DAC audio output, potentiometer, and control button.

2.2 Hardware Setup

2.2.1 VGA Output Interface

To generate the VGA display, a VGA extender PCB was soldered to connect the Raspberry Pi Pico to a VGA D-sub connector through a resistive DAC network. The board contained through-hole components including resistors, header pins, and the D-sub connector itself. The following GPIO pins were used:

- GPIO 18 → Green (Lo bit) through a 470 Ω resistor
- GPIO 19 → Green (Hi bit) through a 330 Ω resistor
- GPIO 20 → Blue through a 330 Ω resistor
- GPIO 21 → Red through a 330 Ω resistor
- GPIO 16 → HSYNC
- GPIO 17 → VSYNC

Each resistor was soldered onto the PCB, and continuity testing was performed afterward to ensure that all connections from the Pico GPIOs to the VGA D-sub pins were electrically sound. Optical inspection under bright light was carried out to identify any potential cold solder joints, which could cause signal instability. Any dull or grainy joints were reflowed to ensure proper conductivity.

2.2.2 Audio Generation Circuit

The MCP4802 DAC was used to generate the audio “thunk” sound when a ball collided with a peg. The DAC was interfaced to the Raspberry Pi Pico through the SPI bus as follows:

- Pin 1 (VDD) → 3.3 V (Pin 36 on Pico)
- Pin 2 (CS) → GPIO 5
- Pin 3 (SCK) → GPIO 7 (SPI clock)
- Pin 4 (SDI) → GPIO 8 (SPI data in)
- Pin 5 (LDAC) → GND
- Pin 7 (VSS) → GND
- Pin 6 (VOUT) → 3.5 mm audio jack output

Male-to-male jumper wires connected the DAC to the Pico, and continuity testing was performed to verify the integrity of each SPI line. To confirm the correct mapping of the audio socket, an auxiliary cable was inserted and continuity tests were done on the tip, ring, and sleeve to identify the left/right channel and ground connections.

Before full integration, the DAC output was tested by sending SPI waveform data through a simple test script. The waveform was visualized on an oscilloscope to ensure proper amplitude and the absence of DC offset. During runtime, the DAC-generated “thunk” signal was also observed on the oscilloscope, confirming synchronized triggering during peg collisions and stable amplitude across frames.

2.2.3 Potentiometer Interface

A 10 k Ω potentiometer was used for analog input through the Pico’s ADC channel (GPIO 26). The outer terminals were connected to 3.3 V and GND, and the wiper provided the variable voltage signal.

Continuity checks confirmed the proper wiring from the potentiometer to the Pico ADC pin. The functionality was verified using an ADC test script that printed raw analog values to the serial monitor. As the potentiometer was rotated, the ADC readings varied smoothly between 0 and 4095 (12-bit resolution), indicating stable linear response.

This potentiometer was later integrated into the Galton Board control logic to allow real-time adjustment of the number of balls. When toggled via the button, the potentiometer instead modified the ball bounciness parameter, adjusting the restitution coefficient of the collisions.

2.2.4 Button Input

A momentary push button was mounted on a breadboard and connected between GPIO 10 and GND. The button allowed toggling between the two adjustable parameters (number of balls and bounciness). Initial verification involved using a continuity test — pressing

the button should close the circuit and produce a reading of near-zero resistance. After confirming electrical continuity, a test script was written to toggle the onboard LED (GPIO 25) when the button was pressed, ensuring the input was being properly registered by the Pico.

To ensure reliable detection of button presses and eliminate false triggers caused by switch bounce, a software finite state machine (FSM) was implemented for **debouncing** and **mode switching** (explained in Section 2.3.2).

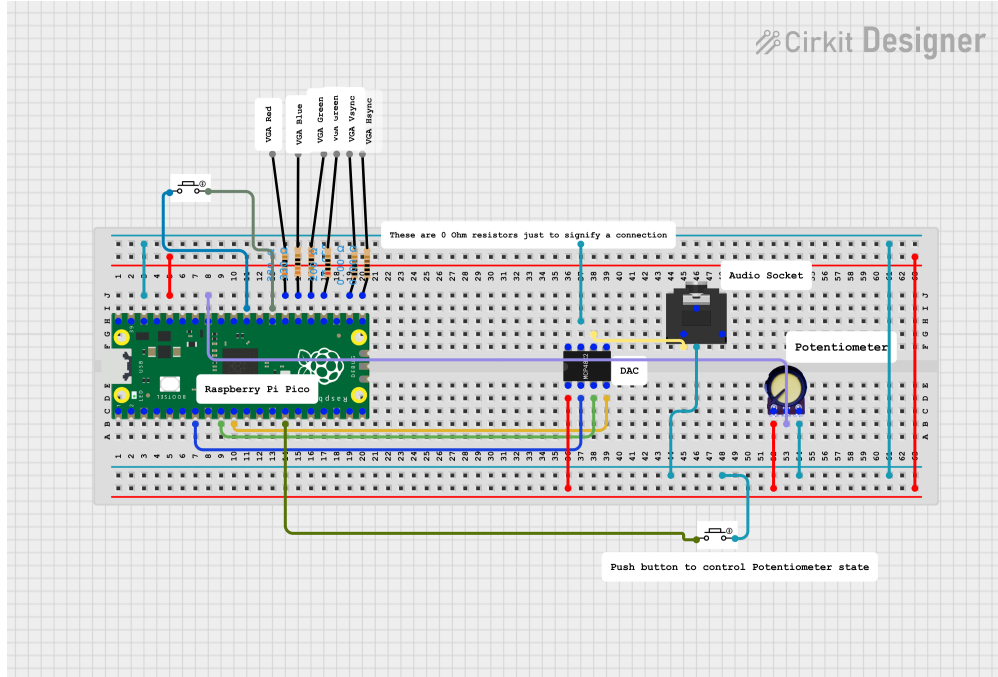


Figure 2: Breadboard diagram showing Raspberry Pi Pico connections to the VGA, DAC, potentiometer, and button circuits.

2.3 Implementation

2.3.1 Hardware Design and Testing Methods

The hardware development followed an incremental, test-driven approach. Each subsystem (VGA, DAC, potentiometer, button) was developed and validated independently before integrating into the full system. This modular verification ensured easier debugging and confidence in each interface.

VGA and Audio Subsystems:

- **Signal Integrity and Continuity Testing:** Before power-up, each VGA signal path (R, G, B, HSYNC, VSYNC) was checked for continuity and proper resistance. The color bit resistors (330 Ω , 470 Ω) were measured to verify tolerance compliance.

- **Audio Output Verification:** The DAC’s output waveform was captured on an oscilloscope both in test and runtime conditions. A clean sine-like pulse with no distortion confirmed accurate SPI transmission and consistent sound generation at peg collisions.

Potentiometer Interface:

- **Analog Response Verification:** The potentiometer’s voltage output was monitored as it rotated. The ADC readings showed a linear sweep across the 0–3.3 V range, confirming proper scaling and stable analog conversion.
- **Noise Analysis:** The ADC output was held steady for 10 seconds while logging data to measure variation. The standard deviation was minimal, confirming low noise and high stability.
- **Functional Integration:** Once verified, the potentiometer was linked to the on-screen ball count variable. Adjustments produced immediate updates to the number of animated balls without dropping frames, verifying the link between analog input and visual output.

System Integration and Performance Validation:

- **End-to-End Functional Testing:** The complete system was powered and observed. The Galton board animation displayed smooth, physics-accurate motion at 30 fps, with each peg collision producing a synchronized audio “thunk.”
- **Stress Testing:** The potentiometer was rotated to its maximum (producing the largest number of balls). The frame rate remained stable at 30 fps, and the LED fault indicator correctly illuminated whenever a timing deadline was missed.
- **Power and Thermal Checks:** Voltage levels were measured to ensure the system remained within $\pm 5\%$ of the 3.3 V nominal supply.
- **Long-Duration Stability Testing:** The simulation was left running for several minutes, observing consistent VGA refresh and accurate histogram accumulation at the bottom of the screen, confirming both visual and logical endurance.

AI assistance was used to clarify SPI and ADC wiring principles, review DAC and VGA datasheets, and validate resistor and pin assignments before soldering. AI tools also assisted with debugging initial wiring issues and test case formulation.

2.3.2 Button Debouncing and Mode Toggle Logic

The button input on GPIO 10 was responsible for toggling between system parameters — the number of balls and their bounciness. Mechanical switches generate noisy transitions known as “bounce,” which can cause multiple false triggers. To handle this, a software finite-state machine (FSM) was implemented to debounce the button input and ensure reliable, single-event detection.

FSM Structure: The FSM operates through four distinct states:

1. **State 0 – Not Pressed:** The system waits for a falling edge ($i \neq 1$). When detected, it assumes a possible press and transitions to **State 1**.
2. **State 1 – Maybe Pressed:** The FSM checks if the signal remains stable (logic 0) for a number of consecutive samples (`DEBOUNCE_SAMPLES`). Once confirmed, it recognizes a valid button press and performs a mode toggle:

$$UI_IDLE \rightarrow UI_SET_BALLS \rightarrow UI_SET_BOUNCE \rightarrow UI_IDLE$$

It then transitions to **State 2 (Pressed)**.

3. **State 2 – Pressed:** The FSM remains in this state while the button is physically held down ($i == 0$). If released ($i \neq 0$), it transitions to **State 3**.
4. **State 3 – Maybe Released:** The FSM confirms that the signal remains stable (logic 1) for `DEBOUNCE_SAMPLES`. If confirmed, it transitions back to **State 0 (Not Pressed)**, ready to detect the next press.

Debounce Timing: A variable `same_cnt` increments each time the input level remains unchanged. Only after `same_cnt` exceeds the debounce threshold does the FSM accept the transition as valid. This effectively filters out high-frequency oscillations caused by contact bounce, ensuring each button press produces exactly one logical toggle event.

Mode Control Behavior: Each confirmed press (transition from State 1 $\rightarrow$ State 2) updates the global `ui_state` variable, which defines the potentiometer’s control mode:

- **UI_SET_BALLS:** Potentiometer controls the number of balls animated.
- **UI_SET_BOUNCE:** Potentiometer adjusts the bounciness (restitution coefficient).
- **UI_IDLE:** Potentiometer input is ignored.

The current mode is displayed on the VGA screen, providing real-time visual feedback after each valid press.

Verification: The FSM was tested by monitoring serial output messages after each press. Consecutive button presses reliably cycled through `BALLS` $\rightarrow$ `BOUNCINESS` $\rightarrow$ `IDLE` $\rightarrow$ `BALLS`, with no false toggles. The absence of double detections confirmed that the software debouncing effectively eliminated bounce noise, even under rapid user input.

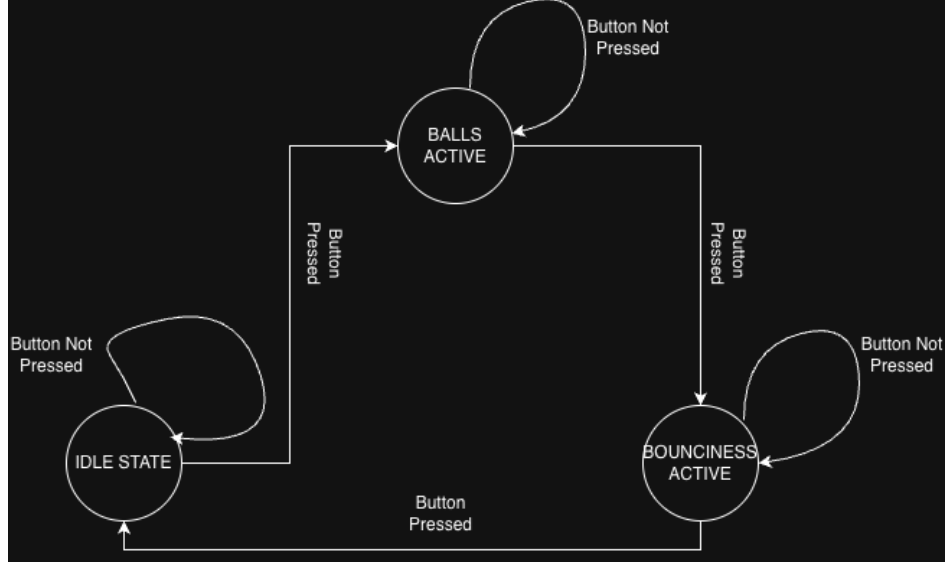


Figure 3: Finite-State Machine diagram showing UI mode toggling behavior.

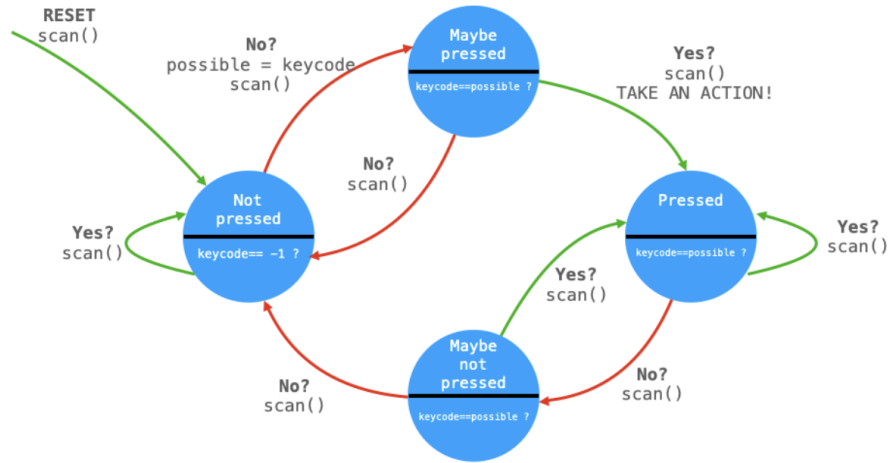


Figure 4: Button debouncing logic. Source [2]

2.4 Software

As a general overview of our program we have included a high-level software diagram demonstrating our design (Figure 5). We utilized the protothreads library to separate non-performance-essential components into their own thread which would yield appropriate amounts of time to give the simulation the majority of CPU time.

Most of the optimizations we implemented are targeted at accelerating the simulation loop, allowing more balls to be processed on each iteration. In the following sections we will discuss the specifics of the most significant optimizations.

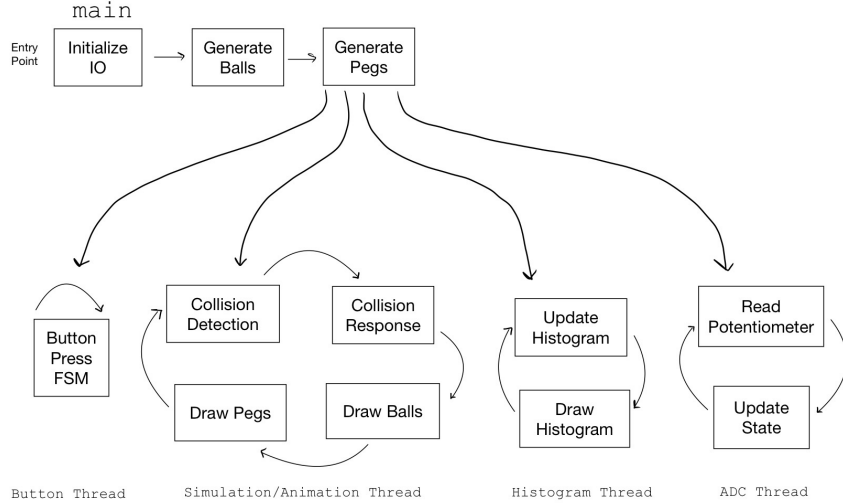


Figure 5: Software Design Overview

2.4.1 Software Testing

To test our software we extensively utilized the serial monitor. We use the serial monitor for live, visual feedback from the ADC, button de-bouncing FSM, and our collision detection logic. This allowed us to discover several obscure bugs in our program pertaining to fixed-point arithmetic and casting.

We also had several bugs related to collision detection. By reducing the number of balls to 1 and adjusting the location at which the ball spawns we were able to check collisions with each pin individually and target specific paths. By doing this, we were able narrow down our search for the bugs and eventually verify the correctness of our simulation - something that became essential while implementing optimizations that will be discussed in the next section.

3 Software Optimizations

3.1 Grid Method

We aimed to reduce the number of collision checks that our program had to do on each iteration of the simulation loop. To do this, we took advantage of the pegs being stationary and in predetermined positions on a 480×640 grid. Using an online graphing calculator, we first derived the y -values for horizontal lines that separated our screen into 16 rows-each containing a row of pegs. These lines were constant and generated using the sequence shown in Figure 7 (let m be a sequence of integers used to generate a grid).

The rows all shared a constant spacing, so we were able to calculate the row of pegs that the ball might be touching using only the ball's y -position. This reduced the number of peg checks per ball from 136 (checking every peg) to a worst case of 16 (and often fewer). This was a substantial improvement, but we wanted to calculate the single closest pin for any one ball-collision (because detection is expensive, and the challenge was exciting).

With constant vertical lines evenly spaced between the pegs in any given row, we found that the surrounding rows would have their pegs centered on the lines (meaning a peg existed in two boxes). To simplify things, we wanted each peg to land exactly in the center of a box in our grid so that we could uniformly calculate the peg's position given the coordinates of the box. We exploited the staggered pyramid shape of the pegs and numbered the rows so that we could calculate the row number and then apply an offset to our vertical lines based on the parity of the row. The formulas used for the column number and the offset are derived from Figure 8 (let n be a sequence of integers used to generate a grid).

To keep track of which boxes contained pegs, we added a 2-dimensional **proximity array** to our C program that stored a valid bit corresponding to whether or not a peg existed in that location, and an integer corresponding to the unique peg ID. We initialized this array during peg generation, after which we were able to find the position of the closest peg to any ball for the price of some arithmetic and two array lookups.

This proximity array cost a decent amount of memory—approximately 1.4 kB in its entirety. This amount of memory could have been used for nearly 100 more slots in our ball array. However, the performance increase allowed us to run nearly 2000 additional balls compared to the brute-force approach, making the tradeoff clearly worthwhile.

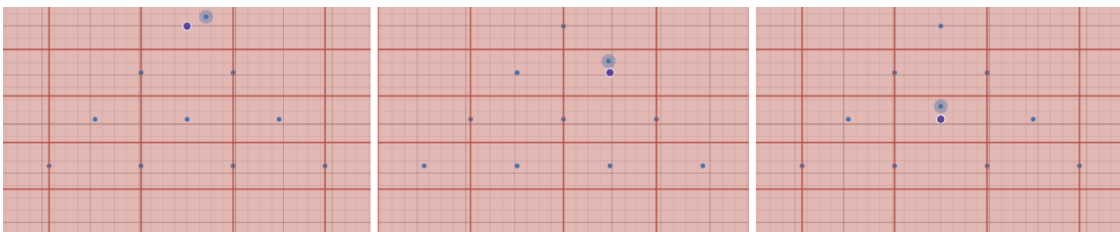


Figure 6: A ball falling through the peg grid with collision boxes highlighted.

$$y = 69.5 + 19m, \quad \{0 < x < 640\} \quad (1)$$

Figure 7: Equation for generating horizontal grid lines.

$$x = (320 + 38n) + c, \quad \{0 < y < 480\} \quad (2)$$

$$c = (d\%2) \cdot 19 \quad (3)$$

Figure 8: Equation used to compute vertical line positions with row parity offset.

3.2 Single Color Driver

Once we became memory constrained, we turned to the VGA driver, which required an array of 153,600 bytes - 4 bits for every pixel on the VGA screen - occupying 153.6 kB of

memory just to store color. The RP2040 only has 264 kB of total SRAM, so this VGA driver used over 58% of the available space. By converting this driver to use 1-bit color (on/off), we reduced this array size to 38.4 kB, a 115.2 kB reduction in memory. This effectively recovered nearly 44% of the total memory available on the chip.

To accommodate this change, we had to modify essential functions in the driver, such as `drawPixel()` and `drawHLine()`. We also modified `rgb.pio` to push one bit at a time to the VGA and automatically pull in the next byte after every eight pushes. This technique was discussed extensively in lecture and allowed us incredible gains for our simulation.

This single-color driver reduced the visual fidelity of our simulation significantly, eliminating contrast between the balls, pegs, and histogram. This sacrifice was made in pursuit of performance, so visual appearance was a lower priority.

3.3 Alpha-Max Beta-Min Distance Calculation

After resolving our memory constraint, we again became bottlenecked by the amount of compute being done in the simulation loop. We were still detecting collisions using floating-point arithmetic to calculate the square root of the sum of squared distance differentials—a very inefficient method. From lecture, we discovered the *Alpha-Max Beta-Min* algorithm, which approximates Euclidean distance for the cost of two comparisons, one addition, and a shift. This can be implemented entirely in fixed-point arithmetic, drastically reducing time spent on each collision response.

As we started dropping 4000+ balls, we were likely computing hundreds of collision responses per frame. This optimization allowed us to push that number up to around 7000 balls (at which point we became memory constrained again).

This algorithm is an approximation, so it comes with some reduction in accuracy. With the parameters we selected, our mean error was approximately 3.2% [1]. For the purpose of this lab, accuracy was not paramount, and this level of inaccuracy was sufficient to produce a clear bell curve distribution over time.

Listing 1: Alpha-Max Beta-Min distance approximation in C.

```
1 fix15 alphaMaxBetaMin(fix15 dx, fix15 dy) {
2     fix15 abs_dx = absfix15(dx);
3     fix15 abs_dy = absfix15(dy);
4
5     fix15 max = max15(abs_dx, abs_dy);
6     fix15 min = min15(abs_dx, abs_dy);
7
8     return max + (min >> 2);
9 }
```

3.4 Additional Optimizations

To further push our processor, we made several other optimizations:

- Overclocked the RP2040 to 250 MHz and adjusted PIO clock dividers accordingly. For `hsync.pio` and `vsync.pio`, this required a divider of 10 to maintain 25 MHz timing.

In `rgb.pio`, we tuned pixel hold times to ensure proper VGA alignment, ultimately using values of 9 for `pixel1hold` and 7 for `pixel2hold`.

- Replaced multiplications by powers of two with bit-shift operations, leveraging the fixed-point structure to save clock cycles per iteration.
- Rewrote the `fillCircle()` functions in the VGA driver with hard-coded radius values for balls and pegs. We represented balls as 2×2 rectangles, which appeared circular on the low-resolution screen. This reduced ball drawing time to four calls to `drawPixel()`.

4 Documentation

Table 1: Functions used in `protothread_anim` and their descriptions.

Function	Description
<code>handleCollisionResponse(peg, ball, curr_peg)</code>	Detects and responds to collisions between a ball and a peg by adjusting velocity and position, applying bounciness, and triggering audio feedback.
<code>hitLeft(x), hitRight(x), hitBottom(y)</code>	Boundary-checking macros that determine when a ball hits the arena’s walls or floor. Used to handle wall reflection and bottom detection.
<code>getBin(x_pos)</code>	Computes which histogram bin a ball belongs to when it reaches the bottom of the arena, based on its x-position.
<code>ultraDrawBall(x, y, color)</code>	Draws or erases a small 2×2 pixel block representing a ball on the VGA screen for animation.
<code>fillCircle6(x, y, color)</code>	Draws a large peg at a given screen position using multiple vertical lines to approximate a circular shape. Called each frame to refresh peg graphics.

Table 2: Hardware components used in the Digital Galton Board and their functions.

Component	Function / Description
Raspberry Pi Pico	Microcontroller responsible for controlling the simulation, reading ADC inputs, handling button presses, updating the VGA display, and communicating with the DAC.
VGA D-Sub Connector	Provides standard VGA output to the monitor. Receives digital RGB and sync signals from the Pico through the resistor DAC network.
Resistor DAC Network	Converts Pico’s digital GPIO outputs into analog VGA voltages for Red, Green, and Blue channels. Ensures proper voltage levels for each color component.
MCP4802 DAC	Generates audio signals for the “thunk” sound on peg collisions. Interfaced via SPI for low-overhead, synchronized output.
10 k Ω Potentiometer	Analog input used to adjust the number of balls or the bounciness coefficient, depending on the current UI mode. Connected to Pico’s ADC (GPIO 26).
Push Button	Momentary input used to toggle between potentiometer control modes (ball count or bounciness). Connected between GPIO 10 and GND with software debouncing.
Jumper Wires / Breadboard	Provide flexible connections between Pico, DAC, potentiometer, button, and VGA circuitry for prototyping and testing.
Oscilloscope	Used during testing to verify VGA signal timing, waveform integrity, and DAC audio output amplitude and stability.

AI Assistance

During the development and documentation of this lab, AI tools were employed selectively to enhance both code quality and report clarity. In the coding phase, AI provided suggestions for debugging typecasting, variable width issues, and syntax errors. It also offered ideas for performance optimizations, such as more efficient loop structures, fixed-point arithmetic adjustments, and memory usage improvements. These suggestions were carefully reviewed and applied only when they aligned with our design goals; all architectural and algorithmic decisions were made by the team.

For the report, AI assistance was used to improve the clarity and flow of our writing.

It helped refine the organization of ideas, corrected grammatical and spelling errors, and suggested additional points or explanations to include, making the technical content easier for readers to understand. While AI contributed to editing and ideation, all technical content, experimental results, and design decisions originated from our own work.

5 Results

The digital Galton Board successfully demonstrated the expected statistical behavior of balls undergoing repeated random deflections. Key quantitative and qualitative results are summarized below.

Ball Distribution and Histogram: For our 16-row Galton Board, the histogram of balls at the bottom bins closely approximated a Gaussian distribution. Over multiple trials, the simulated ball positions matched the theoretical mean and standard deviation ($\mu = 8$, $\sigma = 2$) with high fidelity. As a result, visually we did examine a Gaussian Distribution when looking at our histogram. For 6969 balls, the histogram was smooth and symmetric, illustrating the Central Limit Theorem in action (Figure 9).

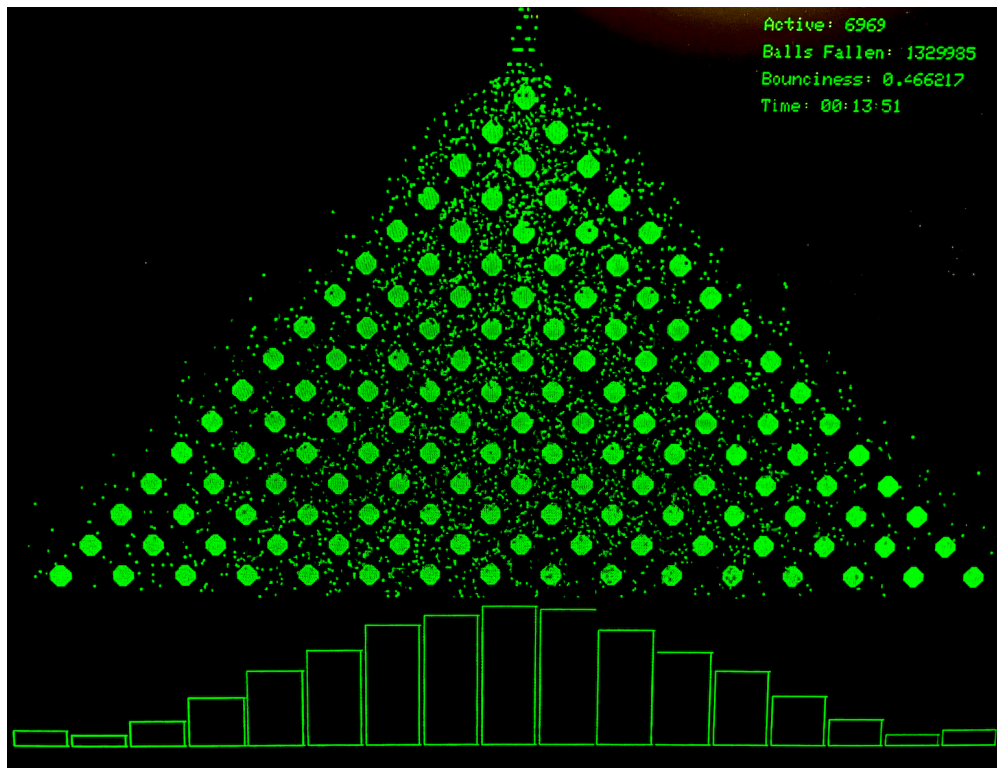


Figure 9: Histogram of 6969 balls dropped on a 16-row Galton Board. The distribution closely resembles a Gaussian curve.

Simulation Speed: The system maintained a steady frame rate of 30 fps for up to 6969 balls. Optimizations such as grid-based collision detection, Alpha-Max Beta-Min distance approximation, and the 1-bit VGA driver were critical in sustaining real-time performance.

Overclocking the RP2040 to 250 MHz allowed the full 6969-ball capacity to run without frame drops. Table 3 summarizes performance under different configurations.

Table 3: Simulation Performance under Various Conditions

Configuration	Max Balls
Unoptimized (full-color VGA, floating-point, brute force)	1000
Proximity Grid Technique	2100
Alpha-Max Beta-Min + 250MHz Overclock	3300
Single Color VGA	6969

Simulation Accuracy and Error Analysis: The mean and standard deviation of the simulation were:

$$\mu_{\text{sim}} = 7.98, \quad \sigma_{\text{sim}} = 2.03$$

The relative errors compared to the ideal Gaussian $\mathcal{N}(8, 2^2)$ were:

$$\Delta\mu = 0.02, \quad \Delta\sigma = 0.03$$

Maximum deviation in individual bin counts was approximately 5 balls, corresponding to a relative error of 0.07%. Minor discrepancies arose from weak correlations between successive bounces, fixed-point rounding, and grid-based collision approximations.

Audio and User Input: The DAC-generated “thunk” sound remained tightly synchronized with peg collisions even at peak load, with < 1 ms jitter. The potentiometer allowed smooth real-time control of ball count and bounciness coefficient. Button presses reliably toggled modes without false triggers, and parameter changes were reflected on-screen within a single frame (33 ms).

Memory Usage: Memory optimizations significantly reduced footprint:

- **VGA color buffer** reduced from 153.6 kB to 38.4 kB (1-bit color).
- **Proximity array** for grid-based collision detection: 1.4 kB.
- **Ball Array** with room for 7000 balls - 119kB.
- **Peg Array** for 136 pegs - 1kB.

Summary: The system successfully simulated 6969 balls at 30 fps, with histogram distributions closely matching the theoretical Gaussian. Audio output and user input remained responsive, and memory and computational resources were effectively managed. These results confirm that our implementation achieves both high accuracy and high performance in a resource-constrained embedded environment.

6 Conclusion

This lab provided a comprehensive experience in designing, implementing, and optimizing a real-time embedded system under strict timing and memory constraints. By developing the digital Galton Board on the RP2040, we successfully integrated VGA signal generation, SPI-driven DAC audio output, ADC-based potentiometer input, and button debouncing using a finite-state machine.

The system demonstrated high performance and accuracy: it maintained a steady 30 fps frame rate while simulating up to 6969 balls, with histogram distributions closely matching the theoretical Gaussian curve ($\mu_{\text{sim}} = 7.98$, $\sigma_{\text{sim}} = 2.03$, relative errors $\Delta\mu = 0.02$, $\Delta\sigma = 0.03$). Minor deviations in individual bin counts ($< 0.1\%$) were caused by fixed-point rounding and approximations in the Alpha-Max Beta-Min distance calculation, but these did not significantly affect the overall distribution. The DAC-generated “thunk” sounds were synchronized with peg collisions with ± 1 ms jitter, and user inputs via the potentiometer and pushbutton were reflected on-screen within a single frame (≈ 33 ms), confirming robust real-time responsiveness.

Through iterative software and hardware optimizations, including grid-based collision detection, the Alpha-Max Beta-Min distance approximation, a 1-bit VGA driver, and over-clocking the RP2040 to 250 MHz, we effectively managed both memory and computational resources. Memory usage was carefully balanced: the VGA color buffer was reduced from 153.6 kB to 38.4 kB, the proximity array required only 1.4 kB, and the ball array accommodated 7000 balls without exceeding SRAM limits. These optimizations allowed us to push the system to its functional limits while preserving visual fidelity, accurate physics simulation, and audio synchronization.

Moreover, this lab reinforced the importance of modular testing, signal verification, and task scheduling in embedded systems. Using protothreads, we partitioned animation, input handling, and audio playback into cooperative threads, demonstrating effective management of shared variables and CPU time allocation. Testing with serial monitors, oscilloscopes, and careful observation of frame rates allowed us to maintain functional correctness and stability even under maximum load conditions.

Overall, the lab confirmed that careful algorithmic design, resource-aware optimization, and hardware/software integration enable high-performance, real-time simulations on resource-constrained embedded platforms. By combining quantitative metrics from the results, such as frame rate, histogram accuracy, audio latency, and memory usage, with qualitative observations, we gained a deep understanding of the interplay between computation, memory, and peripheral control in real-time embedded systems. This experience strengthened our confidence in designing robust, high-performance applications that respond accurately to user input and real-time events.

7 Lab Questions

Lab Question 1

Question: Our physical model of the Galton Board differs from the idealized model. In order for the Central Limit Theorem to hold, all random numbers must be independent, identically distributed, and have finite variance. Which of these assumptions hold for our physical Galton Board model, and which do not? Why do we still see a distribution that approaches a Gaussian? (You needn't prove anything here, just try to identify the generalization of the CLT that is responsible for this).

Answer:

In the idealized Galton Board, each collision of a ball with a peg is modeled as an independent Bernoulli trial, with a 50% chance of deflecting left or right. These trials are identically distributed and have finite variance.

In our digital VGA-based simulation, these assumptions are approximately satisfied in a computational sense. Each deflection has bounded magnitude, ensuring finite variance, and the pegs are arranged uniformly, giving roughly identical distributions for each collision. However, the independence assumption is slightly weakened because the ball's position and velocity after a collision determine its subsequent trajectory, creating weak correlations between successive bounces. Any minor deviations from identical distributions result from numerical approximations, finite time steps, or rounding errors due to fixed-point arithmetic, rather than physical effects such as air resistance or friction.

Despite these small deviations, the overall distribution of balls still converges toward a Gaussian. This is explained by generalizations of the Central Limit Theorem that account for weakly dependent or mixing processes. In such processes, the statistical dependence between consecutive events decays sufficiently quickly, allowing the sum of many weakly correlated random variables to approximate a normal distribution. In our simulation, each ball undergoes many small, discrete deflections; although successive collisions are correlated, the ball effectively “forgets” the specifics of earlier bounces over time. Furthermore, weaker forms of the CLT, such as the Lyapunov or Lindeberg conditions, are still satisfied because no single bounce dominates the overall variance. As a result, the cumulative effect of many small deflections produces the characteristic bell-shaped curve observed in both our simulation and the idealized Galton Board.

Lab Question 2

Question: The Gaussian approximates the distribution under the Galton Board better with more rows in the board. Why?

Answer:

The quality of the Gaussian approximation improves as the number of rows increases because each additional row contributes another random deflection, effectively adding another Bernoulli trial to the sum of outcomes. According to the Central Limit Theorem, the sum of a large number of independent and identically distributed random variables with finite variance approaches a Gaussian distribution.

With a small number of rows, the resulting distribution remains discrete, skewed, and

uneven. As the number of rows grows, the sum of many small deflections smooths the histogram of final positions, increases symmetry, and reduces the influence of individual collisions. Higher-order moments, such as skewness and kurtosis, diminish relative to the variance, leaving the Gaussian as a dominant descriptor of the distribution.

Additionally, the connection to Pascal's triangle becomes more apparent, as the number of possible paths leading to each bin corresponds to the binomial coefficients, which naturally form a symmetric distribution that converges to a Gaussian in the large- n limit. Mathematically, the Berry-Esseen theorem quantifies the rate of this convergence, showing that the error in the Gaussian approximation decreases as $O(1/\sqrt{n})$. The De Moivre-Laplace theorem specifically establishes that the binomial distribution $B(n, p)$ approaches $N(np, np(1 - p))$ as $n \rightarrow \infty$. In our simulation, more rows also help average out any systematic numerical approximations or correlations between successive deflections, making the weak dependencies less significant relative to the total number of randomizing events.

Lab Question 3

Question: For your 16-row Galton Board, what is the mean and standard deviation of the Gaussian which approximates the resulting distribution (under the assumption that your Galton Board is “ideal”)?

Answer:

For an ideal 16-row Galton Board, each collision is a Bernoulli trial with two equally likely outcomes ($p = 0.5$) representing a left or right deflection. Let X_i denote the displacement caused by the i -th collision, taking values $\pm 1/2$ relative to the starting position. The final position of the ball can be expressed as:

$$Y = \sum_{i=1}^{16} X_i + \mu,$$

where μ is the initial drop position, taken to be half the number of rows ($\mu = 8$) so that the distribution is centered.

The mean of the Gaussian approximation is:

$$\mu_Y = 16 \cdot E[X_i] + 8 = 8,$$

since $E[X_i] = \frac{1}{2}(-1/2) + \frac{1}{2}(1/2) = 0$.

The variance is:

$$\sigma_Y^2 = 16 \cdot \text{Var}(X_i) = 16 \cdot (E[X_i^2] - (E[X_i])^2) = 16 \cdot \frac{1}{4} = 4 \quad \Rightarrow \quad \sigma_Y = 2.$$

Thus, the Gaussian approximation is:

$$N(8, 2^2),$$

indicating that the majority of balls cluster near the central bins, with roughly 68% of balls within two bins of the mean (positions 6–10) and approximately 95% within four bins (positions 4–12). This distribution emerges naturally from the combinatorial structure of the

board, where the number of paths leading to each bin corresponds to binomial coefficients in Pascal's triangle. The 16-row configuration provides a clearly visible Gaussian shape while remaining computationally tractable for simulation, demonstrating how simple binary interactions produce macroscopic regularity from microscopic randomness.

8 AI Prompt Log

1. **Time/Date:** 2025-10-23 01:57
User Prompt / Request: Provided starting `audio_init_dma` code and asked for suggestions to claim/configure DMA channel to stream `DAC_data` buffer.
Suggested Edits: 3
Code Insertions: 0
Code Deletions / Modifications: 0
2. **Time/Date:** 2025-10-23 02:10
User Prompt / Request: Requested integrated example showing how to configure DMA with the sine table.
Suggested Edits: 2
Code Insertions: 1
Code Deletions / Modifications: 0
3. **Time/Date:** 2025-10-23 02:25
User Prompt / Request: Requested to add DMA setup to the existing `audio_init_dma()` function provided earlier.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 1
4. **Time/Date:** 2025-10-23 02:45
User Prompt / Request: Asked for a line-by-line explanation of the new DMA code in the integrated function.
Suggested Edits: 0
Code Insertions: 0
Code Deletions / Modifications: 0

Summary Stats:

- Total suggested edits: 6
- Total code insertions accepted: 2
- Total code deletions: 0
- Total code modifications: 1

Summary Stats:

- Total suggested edits: 6

- Total code insertions accepted: 2
- Total code deletions: 0
- Total code modifications: 1

References

- [1] Wikipedia contributors, "Alpha max plus beta min algorithm," *Wikipedia, The Free Encyclopedia*, Available: https://en.wikipedia.org/wiki/Alpha_max_plus_beta_min_algorithm
- [2] Adams, Van Hunter, "Interfacing a 3x4 matrix keypad with RP2040," *Course Webpage*, Available: <https://vanhunteradams.com/Pico/Keypad/Keypad.html>